PROGRAM NO – 22

AIM: To develop a Java program to read a matrix from the console and check whether it is symmetric/not.

PROGRAM

```
import java.util.Scanner;
public class SymmetricMatrix
{
   public static void main(String[] args)
   {
      boolean symmetric=true;
      Scanner s = new Scanner(System.in);
      System.out.print("Enter number of rows: ");
      int rows = s.nextInt();
      System.out.print("Enter number of columns: ");
      int cols = s.nextInt();
      int[][] matrix = new int[rows][cols];
      if (rows != cols)
      {
         System.out.println("The matrix is not symmetric.");
      }
      else
      {
        System.out.println("Enter elements of matrix:");
         for (int i = 0; i < rows; i++)
         {
            for (int j = 0; j < cols; j++)
            {
               matrix[i][j] = s.nextInt();
            }
         }
         for (int i = 0; i < rows; i++)
         {
            for (int j = 0; j < cols; j++)
            {
               if (matrix[i][j] != matrix[j][i])
               {
                     symmetric = false;
                     break;
               }

            }
         }
         if (symmetric)
         {
            System.out.println("The matrix is symmetric.");
         }
```

```
        else
        {
            System.out.println("The matrix is not symmetric.");
        }
    }
  }
}
```

PROGRAM NO – 22.1

AIM: To develop a Java program to read a matrix from the console and print its transpose matrix.

PROGRAM

```
import java.util.Scanner;
public class Transpose
{
  public static void main(String[] args)
  {
    Scanner s = new Scanner(System.in);
    System.out.print("Enter number of rows: ");
    int rows = s.nextInt();
    System.out.print("Enter number of columns: ");
    int cols = s.nextInt();
    int[][] matrix = new int[rows][cols];
    System.out.println("Enter elements of matrix:");
      for (int i = 0; i < rows; i++)
      {
        for (int j = 0; j < cols; j++)
        {
          matrix[i][j] = s.nextInt();
        }
      }
      System.out.println("Transpose of given matrix");
      for (int i = 0; i < rows; i++)
      {
        for (int j = 0; j < cols; j++)
        {
          System.out.print(matrix[j][i]+" ");
        }
        System.out.println();
      }
    }
}
```

PROGRAM NO – 23

AIM: Create CPU class with attribute price. Create an inner class processor (no of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of processor and RAM.

PROGRAM

```
class CPU
{
   double price=15000.0;
   class Processor          // inner class
   {
      // members of nested class
      int cores=10;
      String manufacturer="Pentium";
   }
   static class RAM   // static nested class
   {
      // members of static nested class
      double memory=64;
      String manufacturer="Intel";
   }
}
public class CPUDetails
{
   public static void main ( String[ ] args)
   {
      CPU c = new CPU( );                        // create object of Outer class CPU
      CPU.Processor p=c.new Processor( );      // create an object of inner class Processor using outer class
      CPU.RAM r = new CPU.RAM( );        // create an object of static class RAM using outer class CPU
      System.out.println("CPU class");
      System.out.println("-------------");
      System.out.println("CPU Price = " + c.price );
```

```
        System.out.println("Processor inner class");
        System.out.println("-----------------------");
        System.out.println("No. of cores = " +p.cores);
        System.out.println("Manufacturer = " +p.manufacturer);
        System.out.println("RAM static nested class");
        System.out.println("-----------------------");
        System.out.println("Memory = " +r.memory);
        System.out.println("Manufacturer = " +r.manufacturer);
    }
}
```

PROGRAM NO – 24

AIM: To develop a Java program to to perform complex number addition.

PROGRAM

```
public class ComplexNumber
{
   double real,img ;                          // variables for real and imaginary parts of complex numbers
   ComplexNumber(double r,double i)           //constructor to initialize the complex number
   {
      real=r;
      img=i;
   }
   public static ComplexNumber sum(ComplexNumber c1,ComplexNumber c2)
   {
      ComplexNumber temp = new ComplexNumber(0,0) ;     //creating a temporary complex number to hold the sum
      temp.real=c1.real+c2.real;
      temp.img=c1.img+c2.img ;
      return temp ;
   }
   public static void main ( String args [ ] )
   {
      ComplexNumber c1 = new ComplexNumber( 5.5 , 4 ) ;
      ComplexNumber c2 = new ComplexNumber ( 1.2 , 3.5 ) ;
      ComplexNumber temp = sum( c1 , c2 ) ;
      System.out.printf( "Sum is : "+ temp . real+" + "+ temp . img +" i " ) ;
   }
}
```

PROGRAM NO – 25

AIM: To develop a Java program to sort strings

PROGRAM

```
import java.util.Arrays;
import java.util.Scanner;
public class SortString
{
   public static void main(String[] args)
   {
      Scanner s = new Scanner(System.in);
      String strElement;
      System.out.println("Enter the number of Strings: ");
      int n = s.nextInt();
      s.nextLine();
      String str[] = new String[n];
      System.out.println("Enter the Strings: ");
      for (int i = 0; i < n; i++)
      {
         str[i] = s.nextLine();  // Reading string values
      }
      Arrays.sort(str);                    // sorts the array "str" in ascending order
      System.out.println("\nSorted Strings: ");
      for(String word:str)   // Stores each string in "str" array into "word" during each iteration
      {
         System.out.println(word);
      }
      s.close();
   }
}
```

NOTES

“ for(String word:str) ” is an enhanced for-loop (also called a "for-each" loop) in Java. It is used to iterate over arrays or collections in a more readable way compared to a traditional `for` loop.

- String word → A temporary variable that holds each element (string) from the str array during each iteration.
- : str → The array str (which contains user-inputted strings) that we are looping through.

How It Works in Your Program:

1. The array str contains n strings entered by the user.
2. **Arrays.sort(str);** sorts the strings in ascending order (alphabetically).
3. The for-each loop:
    - Takes each string from the sorted str array **one by one**.
    - Stores it in the variable word.
    - Prints word.

PROGRAM NO – 26

AIM: To develop a Java program to search a number from a given set of numbers.

PROGRAM

```
import java.util.Scanner;
public class SearchNum
{
  public static void main(String[] args)
  {
    int i,pos=0,flag=0;
    Scanner s = new Scanner(System.in);
    System.out.println("Enter the number of elements: ");
    int n = s.nextInt();
    int ar[] = new int[n];
    System.out.println("Enter the elements: ");
    for(i = 0; i<n; i++)
    {
      ar[i] = s.nextInt();
    }
    System.out.println("Enter the element to be searched: ");
    int e = s.nextInt();
    for(i=0;i<n;i++)
    {
      if(ar[i]==e)
      {
        flag=1;
        pos=i+1;
        break;
      }
    }
    if(flag!=0)
    {
      System.out.println(e + " is found at position-"+pos);
    }
```

```
  else
      {
          System.out.println(e + " is NOT found!!");
      }
    }
}
```

AIM: To develop a Java program to search a word from a given set of words.

PROGRAM

```
import java.util.Scanner;
public class SearchString
{
   public static void main(String[]args)
   {
      int i,j,pos=0;
      boolean state=false;
      Scanner s=new Scanner(System.in);
      System.out.println("Enter number of elements in an array");
      int num=s.nextInt();
      String word[]=new String[num];
      System.out.println("\nEnter "+num+" words");
      s.nextLine();
      for(i=0;i<num;i++)
      {
        word[i]=s.nextLine();
      }
      System.out.println("Enter element to be searched : ");
      String search=s.nextLine();
      for(i=0;i<num;i++)
      {
         if(word[i].equals(search))
         {
            pos=i+1;
            state=true;
            break;
         }
      }
      if(state)                          // if state=true
      {
         System.out.println("Element found at position="+(pos));
      }
      else
      {
         System.out.println("Element not found");
      }
   }
}
```

AIM: To develop a Java program to perform string manipulation.

PROGRAM

```
import java.util.Scanner;
public class StringManip
{
   public static void main(String[]args)
   {
      Scanner sc=new Scanner(System.in);
      System.out.println("Enter the string");
      String str1=sc.nextLine();
      System.out.println("Length of string="+str1.length());
      System.out.println("Character at 2nd position="+str1.charAt(1));
      System.out.println("String contains 'ple' sequence:"+str1.contains("ple"));
      System.out.println("String ends with e:"+str1.endsWith("e"));
      System.out.println("Replace 'pp' with 'nk':"+str1.replaceAll("pp","nk"));
      System.out.println("Lower case:"+str1.toLowerCase());
      System.out.println("Upper case:"+str1.toUpperCase());
      System.out.println("Index of 'e':"+str1.indexOf('e'));
      System.out.println("Substring from index 2 :"+str1.substring(2));
      System.out.println("Enter 2nd string");
      String str2=sc.nextLine( );
      System.out.println("String 1 concatenated with String 2 : "+str1.concat(str2));
   }
}
```

PROGRAM NO – 29

AIM: To develop a Java program to create a class for Employee having attributes eNo, eName,eSalary. Read n employee information and search for an employee using given eNo, using the concept of array of objects.

PROGRAM

```
import java.util.Scanner;
class Employee
{
  int eNo;
  String eName;
  double eSalary;
  Employee(int n, String name, double sal)
  {
    eNo = n;
    eName = name;
    eSalary = sal;
  }
}
class EmpSearch
{
  public static void main(String[] args)
  {
    Scanner s = new Scanner(System.in);
    System.out.print("Enter the number of employees: ");
    int n = s.nextInt();
    Employee e[] = new Employee[n];          //e[] - Array of objects
    for (int i = 0; i < n; i++)
    {
      System.out.println("\nEnter details for employee " + (i + 1) + ":");
      System.out.print("Employee Number: ");
      int eno = s.nextInt( );
      System.out.print("Employee Name: ");
      String name = s.next( );
      System.out.print("Employee Salary: ");
      double salary = s.nextDouble();
      e[i] = new Employee(eno, name, salary);
    }
    System.out.print("\nEnter the employee number to search: ");
    int searchEno = s.nextInt();
    int flag = 0;
```

```
        for (Employee emp : e)
        {
            if (emp.eNo == searchEno)
            {
                System.out.println("Employee found:");
                System.out.println("Employee Number: " + emp.eNo);
                System.out.println("Employee Name: " + emp.eName);
                System.out.println("Employee Salary: " + emp.eSalary);
                flag = 1;
                break;
            }
        }


    if (flag == 0)
        {
            System.out.println("Employee with Employee Number " + searchEno + " not found.");
        }
    }
}
```

NOTES

"for (Employee emp : e)"

- It is an **enhanced for loop** (also known as a "for-each" loop) in Java. It is used to iterate over arrays or collections in a more readable and concise way compared to a traditional `for` loop.
- `Employee emp` → A variable of type `Employee`, which will store each element of the array `e` one by one.
- `e` → The array `e` (which contains `Employee` objects) that we are iterating over.
- `e` is an array of `Employee` objects.
- The loop automatically assigns each element (employee object) of `e` to the variable `emp` one at a time.
- Inside the loop, you can access `emp.eNo`, `emp.eName`, and `emp.eSalary` for each `Employee` object.

PROGRAM NO – 29.1

AIM: To develop a Java program to create a class for Students having attributes sNo, sName,marks. Read “n” student information and search for a student using given eNo, using the concept of array of objects.

PROGRAM

Leave same space as was needed for Pgm-29

PROGRAM NO – 30

AIM: To develop a Java program to calculate **area of different shapes using overloaded functions**

PROGRAM

```
import java.util.Scanner;
public class Shapes
{
   void area(int r1)            // Overloaded function definition
   {

      System.out.println("\nArea of Circle with Radius "+r1+" = "+3.14*r1*r1);
   }
   void area(int l1,int b1)
   {
      System.out.println("\nArea of Rectangle with dimensions "+l1+" X "+b1+" ="+l1*b1);
   }
   void area(int l2,int b2,int h2)
   {
      System.out.println("\nArea of Cuboid is with dimensions "+l2+" X "+b2+" X "+h2+" =
"+l2*b2*h2);
   }
   public static void main(String[] args)
   {
      Scanner s = new Scanner(System.in);
      System.out.println("\nEnter the Length");
      int l = s.nextInt();
      System.out.println("Enter the Breadth");
      int b = s.nextInt();
      System.out.println("Enter the Height");
      int h = s.nextInt();
      System.out.println("Enter the Radius");
      int r = s.nextInt();
      Shapes ob = new Shapes();
      ob.area(r);       //Calling overloaded method area() with different no. parameters
      ob.area(l,b);
      ob.area(l,b,h);
   }
}
```

Notes

Polymorphism

Polymorphism in Java is a concept by which we can perform a single action in different ways. There are two types of polymorphism in Java:

1) Compile-time polymorphism
2) Run-time polymorphism.

We can perform polymorphism in Java by:
1. Method Overloading,
2. Method Overriding,
3. Dynamic method dispatch, or
4. Using interfaces (will be covered in next chapter)

3.1 Method Overloading in Java

Method overloading occurs when two or more methods within a class share same name with different parameter declaration i.e., a class has multiple methods having same name but differ in their parameters. When an overloaded method is invoked, the type and/or number of arguments determine which version of the overloaded method needs to be actually called. Thus, overloaded methods must differ in the type of parameters and/or the number of parameters. Method overloading increases the readability of the program.

There are two ways to overload the method in Java:
1. using different number of arguments
2. using different data type for arguments
Note: Difference in return type alone is insufficient to distinguish between various overloaded methods.

PROGRAM NO – 31

AIM: To develop a Java program to find sum of different numbers(double, int) using overloaded functions

Leave same space as was needed for Pgm-30

PROGRAM NO – 32

AIM: To develop a Java program to demonstrate single Inheritance. Create a class InheritanceDemo (rollno, name) which is inherited by ChildInheritance (marks, display()).

PROGRAM

```
class InheritanceDemo      //Parent class
{
   int rollno=10;
   String name="Anitha";
}
class ChildInheritance extends InheritanceDemo            //Child class ChildInheritance
{
   int marks=80;
   void display()
   {
      System.out.println("Roll no.:"+rollno);
      System.out.println("Name.:"+name);
      System.out.println("Marks:"+marks);
   }
}
class SingleInheritance
{
   public static void main(String args[])
   {
      ChildInheritance ob=new ChildInheritance();
      ob.display();
   }
}
```

NOTES

Inheritance

- Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. This way Java supports hierarchical classification.
- Inheritance represents IS-A relationship which is also known as a parent-child relationship.
- Using inheritance, new classes are built based on existing classes. When we inherit from an existing class, we can reuse methods and fields of the parent class. We can also add new methods and fields in our current class.
- Thus, inheritance interacts with encapsulation very well. If a given class encapsulates some attributes, then any subclass i.e., the inherited class, will have the same attributes plus any that it adds as part of its specialization.
- Without inheritance, each object would need to define all its characteristics explicitly. Thus, using inheritance considerably reduces program complexity.

"We create objects only for child classes."

Terms used in Inheritance

• **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

• **Sub Class**: Subclass is a class which inherits the other class. It is also referred to as a derived class, extended class, or child class.

• **Super Class**: Super class is the class from where a subclass inherits the features. It is also called a base class or a parent class.

Syntax of Java Inheritance

- To inherit a class, we incorporate the definition of one class into another using **extends** keyword, as given below.
- Here, Subclass-name inherits the methods and fields from Superclass-name. The extends keyword indicates creation of a new class that derives from an existing class. The meaning of "extends" is to increase the functionality of the super-class.

```
class Superclass-name
{
    //methods and fields
}
class Subclass-name extends Superclass-name
{
    //methods and fields specific to subclass
}
```

Types of inheritance in java

Java supports three types of inheritance:

1) Single
2) Multilevel
3) Hierarchical inheritance.

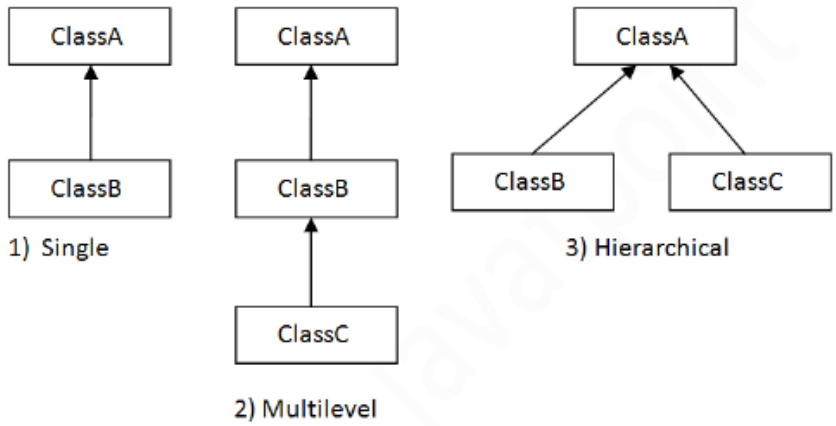
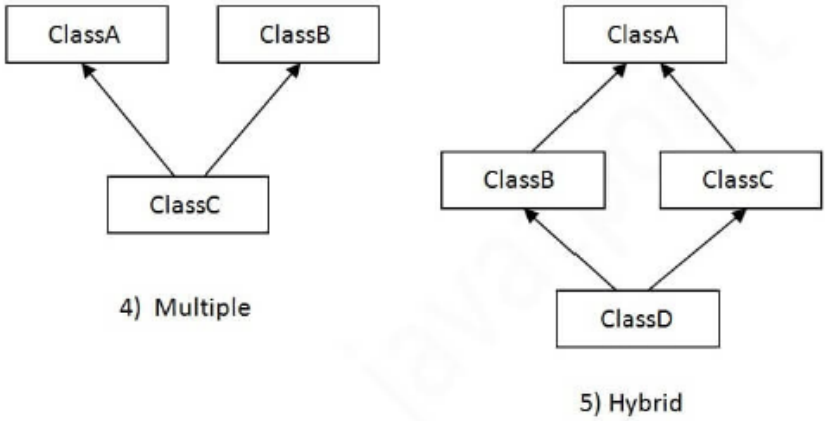


Figure 2.2: Types of inheritance supported by Java



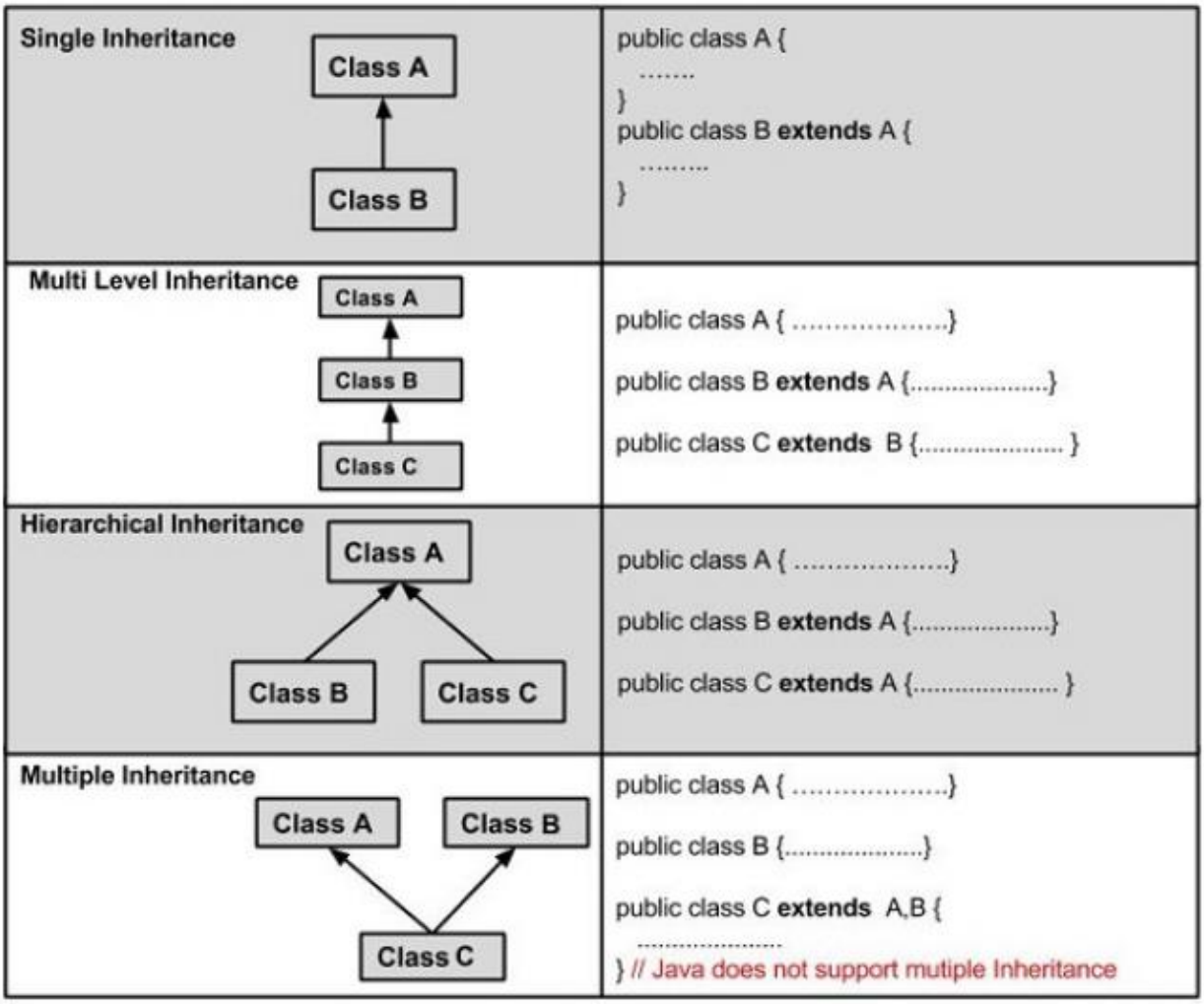


Figure 2.4: Types of inheritance: a recap

1) Single Inheritance

When a class inherits another class, it is known as a single inheritance.

Eg : Dog class inherits the Animal class, so there is the single inheritance.

```
class Animal
{
        void eat ( )
        {
                System.out.println ( " e a t ing . . . " ) ;
        }
}
class Dog extends Animal
{
        void bark ( )
        {
                System.out.p r i n t l n ( " barking . . . " ) ;
        }
}
class TestInheritance
{
        public static void main ( String args [ ] )
```

```
        {

                Dog d = new Dog ( ) ;
                d . bark ( ) ;
                d . eat ( ) ;
        }
}
```

Output

barking...
eating...

PROGRAM NO – 33

AIM: To develop a Java program to demonstrate single Inheritance. Create a class Inheritance(id, ename) which is inherited by the class "Child" (age,putdata())

Leave same space as was needed for Pgm-32